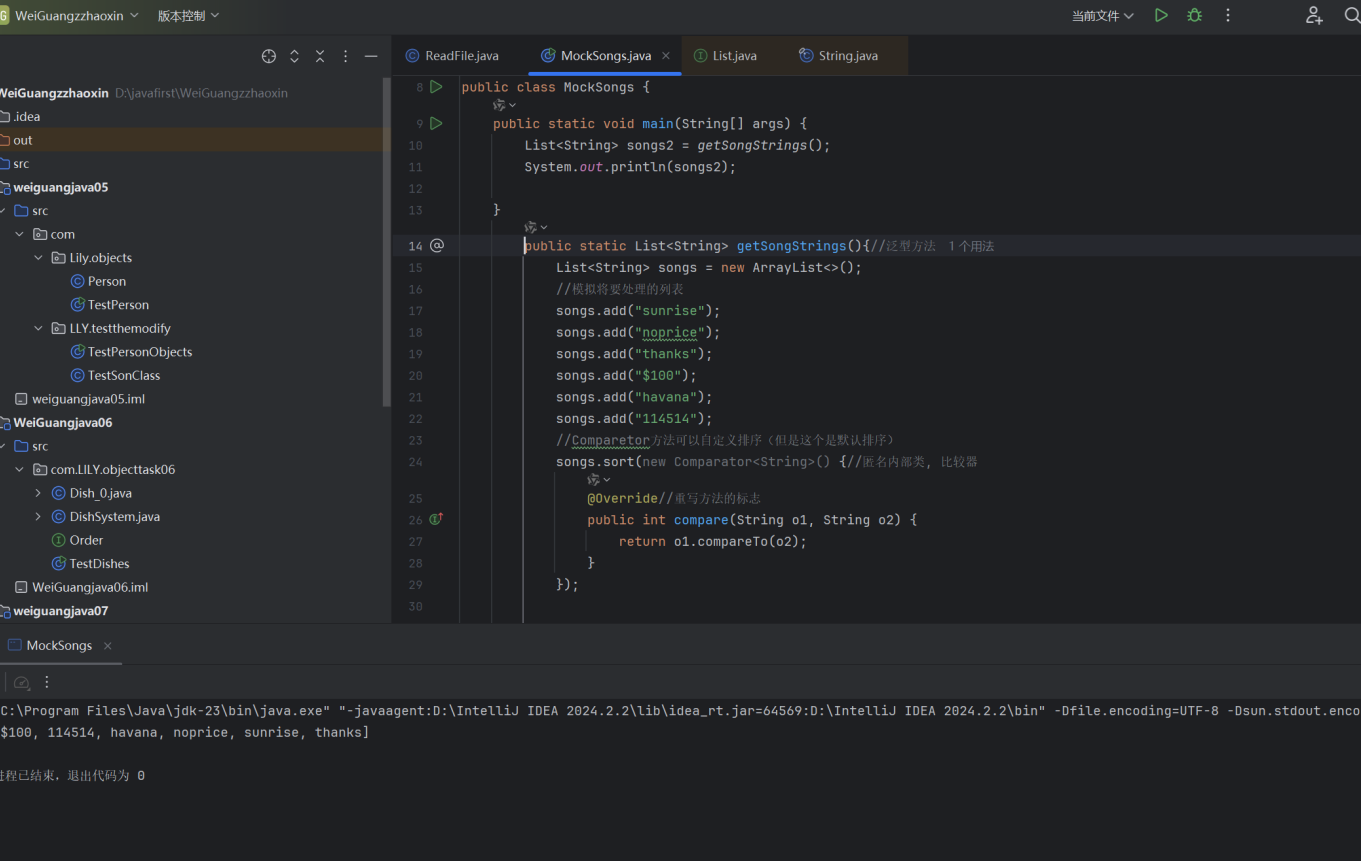


微光招新

Java 08

Task1

1. 如图



```
ReadFile.java MockSongs.java x List.java String.java
7
8 public class MockSongs {
9     public static void main(String[] args) {
10         List<String> songs2 = getSongStrings();
11         System.out.println(songs2);
12     }
13
14
15
16     public static List<String> getSongStrings(){//泛型方法 1个用法
17         List<String> songs = new ArrayList<>();
18         //模拟将要处理的列表
19         songs.add("sunrise");
20         songs.add("noprice");
21         songs.add("thanks");
22         songs.add("$100");
23         songs.add("havana");|
24         songs.add("114514");
25         Collections.sort(songs);//这个是默认的排序方法，按照字母顺序排序和数据大小排序
26
27         return songs;
28     }
29
30
```

sort的源码：

```
自: 1.8
@Contract(mutates = "this")
/unchecked, rawtypes/
default void sort(Comparator<? super E> c) {
    Object[] a = this.toArray();
    Arrays.sort(a, (Comparator) c);
    ListIterator<E> i = this.listIterator();
    for (Object e : a) {
        i.next();
        i.set((E) e);|
    }
}
```

Task2

```

public class TestSong {
    public static void main(String[] args) { //定义对象
        Song s1=new Song( title: "DeartRose", artist: "Faouzia", bpm: 80);
        Song s2=new Song( title: "StayinSlience", artist: "star1", bpm: 70);
        Song s3=new Song( title: "Song3", artist: "star2", bpm: 90);
        Song s4=new Song( title: "Song4", artist: "star3", bpm: 100);
        Song s5=new Song( title: "Song5", artist: "star4", bpm: 110);
        Music m1=new Music( name: "sing", actor: "actor1", id: 1);
        Music m2=new Music( name: "dance", actor: "actor2", id: 2);
        Music m3=new Music( name: "rap", actor: "actor3", id: 3);
        Music m4=new Music( name: "pop", actor: "actor4", id: 4);
        List<Song> songs2=new ArrayList<>();
        songs2.add(m1); //如果泛型中写的是Song, 那么只能添加Song类型的对象, 添加Music对象会报错
        songs2.add(s1);
        songs2.add(s2);
        songs2.add(s3);
        songs2.add(s4);
        songs2.add(s5);
    }
}

```

```

public static void main(String[] args) { //定义对象
    Song s1=new Song( title: "DeartRose", artist: "Faouzia", bpm: 80);
    Song s2=new Song( title: "StayinSlience", artist: "star1", bpm: 70);
    Song s3=new Song( title: "Song3", artist: "star2", bpm: 90);
    Song s4=new Song( title: "Song4", artist: "star3", bpm: 100);
    Song s5=new Song( title: "Song5", artist: "star4", bpm: 110);
    Music m1=new Music( name: "sing", actor: "actor1", id: 1);
    Music m2=new Song( title: "dance", artist: "actor2", bpm: 2);
    Music m3=new Song( title: "rap", artist: "actor3", bpm: 3);
    Music m4=new Song( title: "pop", artist: "actor4", bpm: 4);
    List<? super Music> songs2=new ArrayList<>();
    songs2.add(m1); //但是如果Music是Song的父类, 那么可以通过通配符添加
    songs2.add(m2); //也可以添加由多态定义的实际类型为song的对象
    songs2.add(s1);
    songs2.add(s2);
    songs2.add(s3);
    songs2.add(s4);
    songs2.add(s5);
}

```

```
public static void main(String[] args) { //定义对象
    Song s1=new Song( title: "DeartRose", artist: "Faouzia", bpm: 80);
    Song s2=new Song( title: "StayinSlience", artist: "star1", bpm: 70);
    Song s3=new Song( title: "Song3", artist: "star2", bpm: 90);
    Song s4=new Song( title: "Song4", artist: "star3", bpm: 100);
    Song s5=new Song( title: "Song5", artist: "star4", bpm: 110);
    Music m1=new Music( name: "sing", actor: "actor1", id: 1);
    Music m2=new Song( title: "dance", artist: "actor2", bpm: 2);
    Music m3=new Song( title: "rap", artist: "actor3", bpm: 3);
    Music m4=new Song( title: "pop", artist: "actor4", bpm: 4);
    List<Object> songs2=new ArrayList<>();
    songs2.add(m1); //如果是Object类型，可以添加任意类型
    songs2.add(m2);
    songs2.add(s1);
    songs2.add(s2);
    songs2.add(s3);
    songs2.add(s4);
    songs2.add(s5);
}
```

ovetest 8 个问题

泛型既可以使代码更灵活：（1）？通配符可以表示任意类型（比如List<?>可以接收任何类型的数据）；（2）同时，泛型类型会在编译时全部被擦除，变成Object类型，（所以不支持基本数据类型）（3）泛型类与方法定义时可以不不确定参数类型，等需要创建对象时自动指认，这也表明泛型类和泛型方法可以重复利用；泛型还可以使代码更安全：编译器会检查类型兼容与否，防止类型转化错误；泛型可以规定输入参数的类型，防止错误输入。